SMART AGRO

PROBLEM STATEMENT

The agricultural sector faces dual challenges: the need for sustainable energy and efficient water management. Traditional farming often relies on fossil-fuel-powered irrigation and lacks real-time environmental monitoring, leading to resource wastage and lower crop yields. Furthermore, land used for solar farms often competes with agricultural land rather than complementing it.

PROJECT OVERVIEW

The **Smart Agro-Solar System** is an integrated solution that combines Agrivoltaics (solar energy production on agricultural land) with an automated, IoT-based irrigation system. By installing solar panels above crops, the system generates clean energy while simultaneously providing a controlled environment for plant growth. The integrated IoT framework ensures optimal water usage based on real-time soil and weather data.

SOLUTION OFFERED

The system comprises three primary functional layers:

- **Agrivoltaic Layer:** Strategically placed solar panels that provide shade to crops, reducing evaporation and protecting plants from extreme heat while generating electricity.
- **IoT Monitoring Layer:** Utilizes sensors to track soil moisture, temperature, and humidity. This data is processed to automate irrigation, ensuring water is only used when necessary.
- **Data & Control Layer:** A centralized dashboard (often mobile or web-based) that allows farmers to monitor energy production and crop health remotely.

END USERS

- Individual Farmers: Seeking to reduce electricity and water costs while increasing yield.
- Agricultural Cooperatives: Aiming for large-scale sustainable farming practices.
- Smart Village Initiatives: Using solar energy to power local infrastructure alongside farming.

TECHNOLOGY USED:

- **Hardware (IoT):** ESP32 , Arduino microcontrollers, DHT11 (Temperature & Humidity) sensors, and Soil Moisture sensors.
- **Energy Components:** Photovoltaic (PV) solar panels, Lithium-ion battery storage, and charge controllers.
- **Software & Connectivity:**
    - **Programming:** C++ for microcontroller logic.
    - **Backend/Cloud:** Integration with platforms like Firebase or AWS for data logging.
    - **Frontend:** A user interface developed with React or Flutter for real-time monitoring.

Real-Time IoT Ingestion

```python
@app.route("/api/sensor-ingest", methods=["POST"])
def sensor_ingest():
    global latest_pi_data
    try:
        data = request.get_json()

        latest_pi_data['temperature'] = data.get('temperature', latest_pi_data['temperature'])
        latest_pi_data['humidity'] = data.get('humidity', latest_pi_data['humidity'])
        latest_pi_data['moisture'] = data.get('soil', data.get('moisture', latest_pi_data['moisture']))
        latest_pi_data['irradiance'] = data.get('solar_output', data.get('irradiance', latest_pi_data['irradiance']))
        latest_pi_data['light'] = data.get('light_raw', data.get('light', latest_pi_data['light']))

        print(f"📡 [HTTP INGEST] {latest_pi_data}")

        # Store in history
        sensor_history.append({
            **latest_pi_data,
            "timestamp": datetime.now().isoformat()
        })
        if len(sensor_history) > 500:
            sensor_history.pop(0)

        # Emit to all connected dashboards
        socketio.emit('sensor-update', {
            "temperature": latest_pi_data['temperature'],
            "humidity": latest_pi_data['humidity'],
            "soil": latest_pi_data['moisture'],
            "solar_output": latest_pi_data['irradiance'],
            "light_raw": latest_pi_data['light'],
            "timestamp": datetime.now().isoformat()
        })

        return jsonify({"success": True}), 200

    except Exception as e:
        print(f"Sensor ingest error: {str(e)}")
        return jsonify({"error": str(e)}), 500
```

- **Function:** /api/sensor-ingest
- **What it does:** Acts as the high-speed bridge between the field sensors and the web dashboard.
- **Key Technical Points:**
    - **Data Reception:** Instantly receives JSON payloads (temperature, humidity, moisture, light) forwarded by the Raspberry Pi edge gateway.
    - **Memory Management:** Efficiently maintains a rolling buffer of the last 500 readings in the server's RAM for quick historical charting.
    - **Zero-Latency Updates:** Uses WebSockets (socketio.emit) to push new data directly to the React frontend in milliseconds, eliminating the need for the user to refresh the page.